Operating Systems Technologies

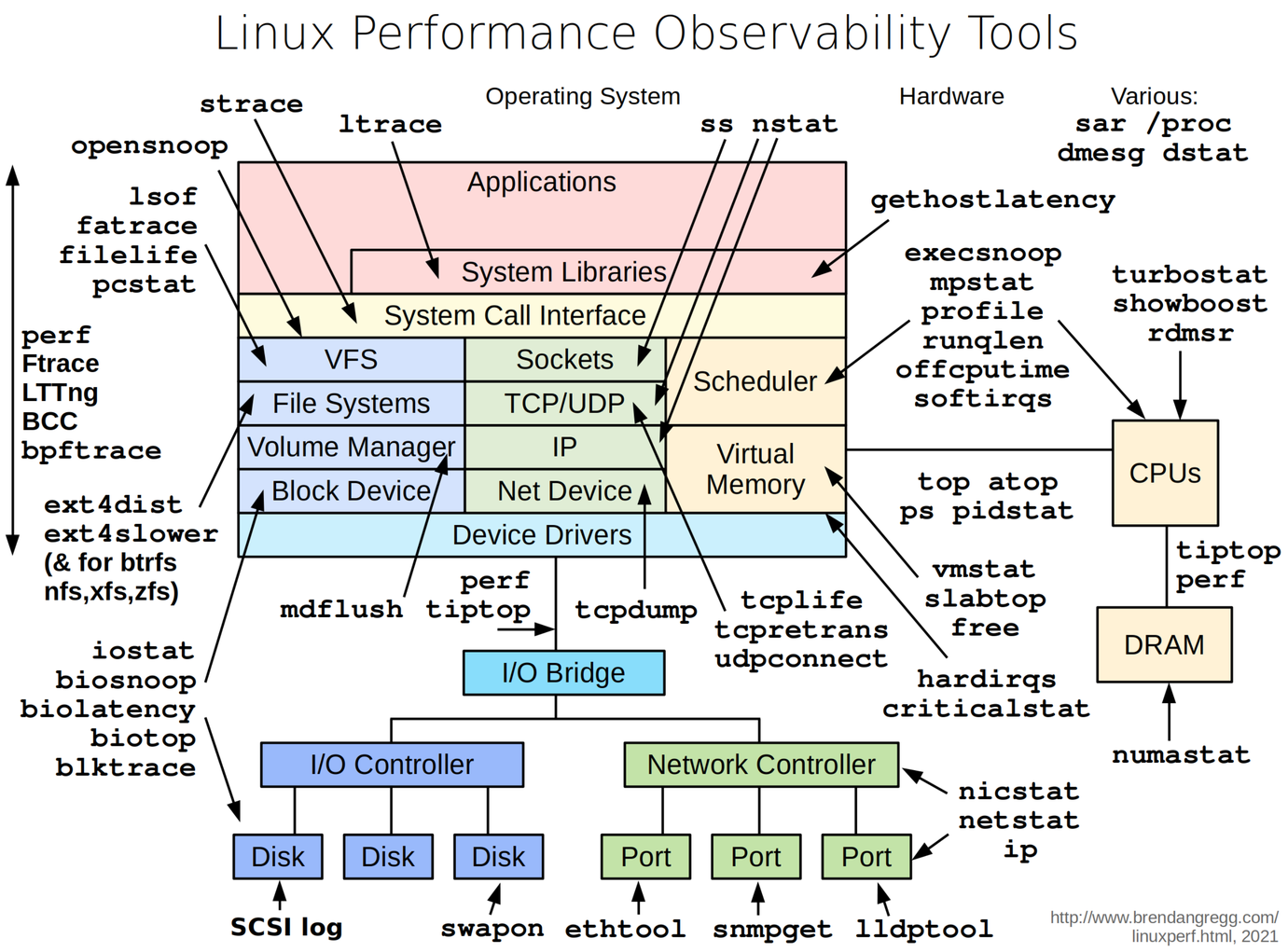
(Tecnologias de Sistemas Operativos)

eBPF

University of Minho

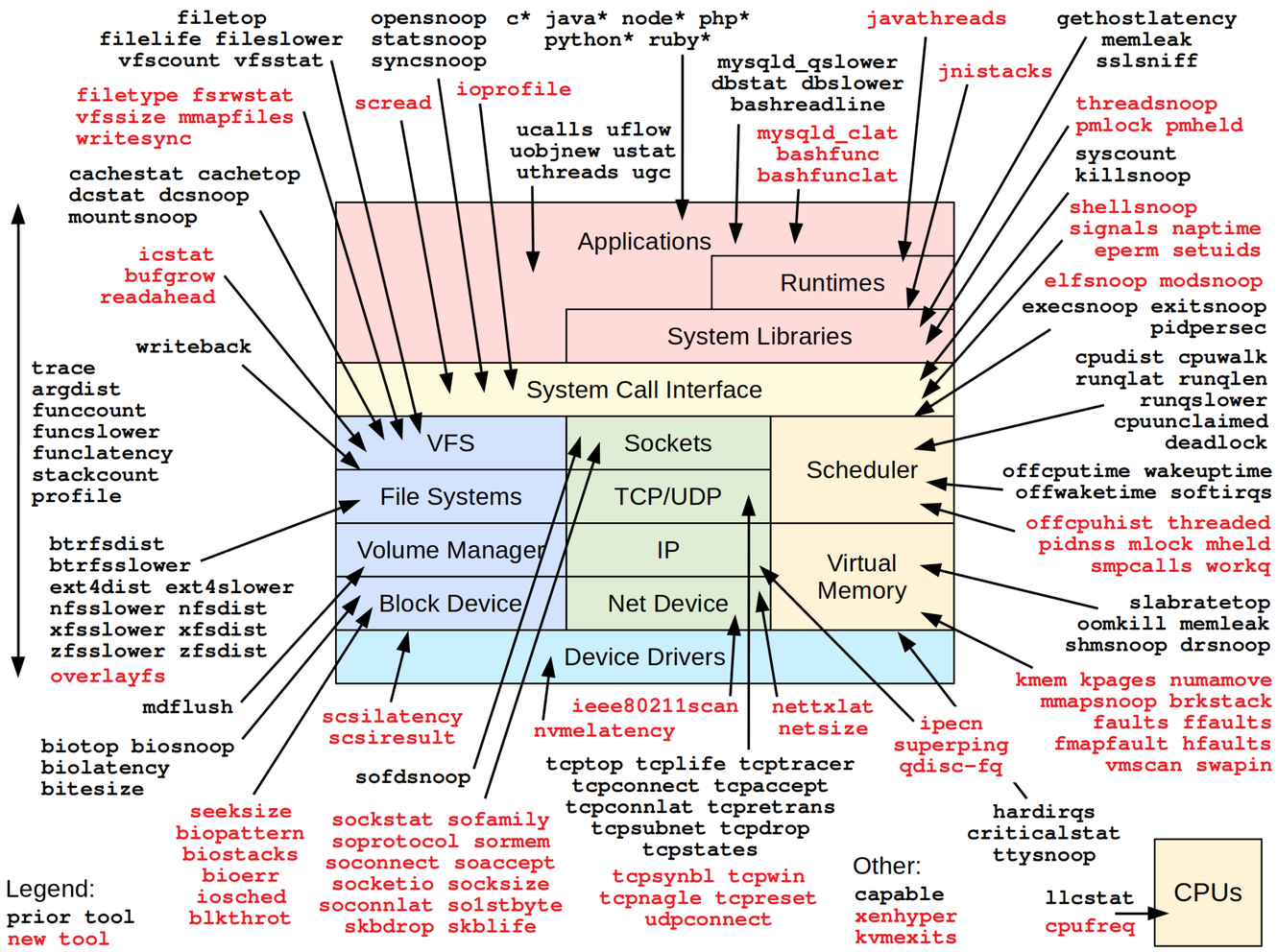
2025-2026

Observability



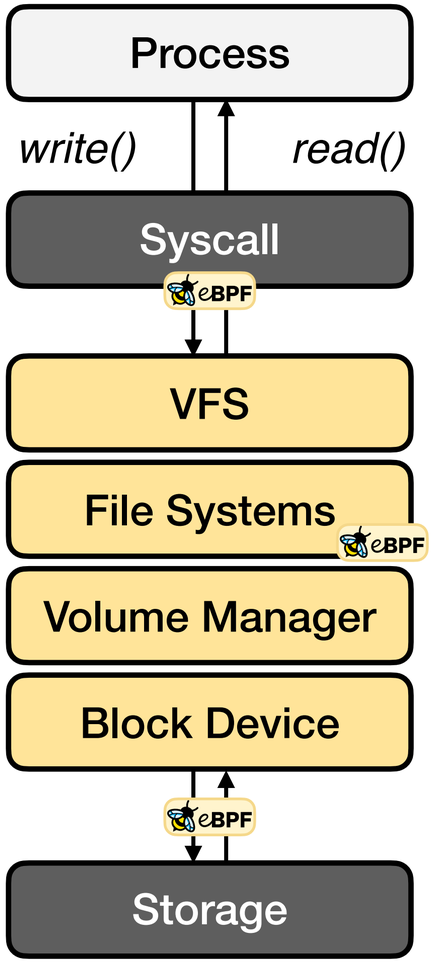
Observability with eBPF

New tools developed for the book **BPF Performance Tools: Linux System and Application Observability** by Brendan Gregg (Addison Wesley, 2019), which also covers **prior BPF tools**



What is eBPF?

- The **extended Berkley Packet Filter (eBPF)** is a kernel technology that allows developers to run custom programs directly inside the OS kernel.
  - Executes small, sandboxed programs in a privileged kernel context.
  - Triggered by events (e.g., networking, disk I/O, syscalls).
  - Allows extending kernel capabilities without modifying kernel source code or loading custom kernel modules.
  - Ensures the safety of the kernel.



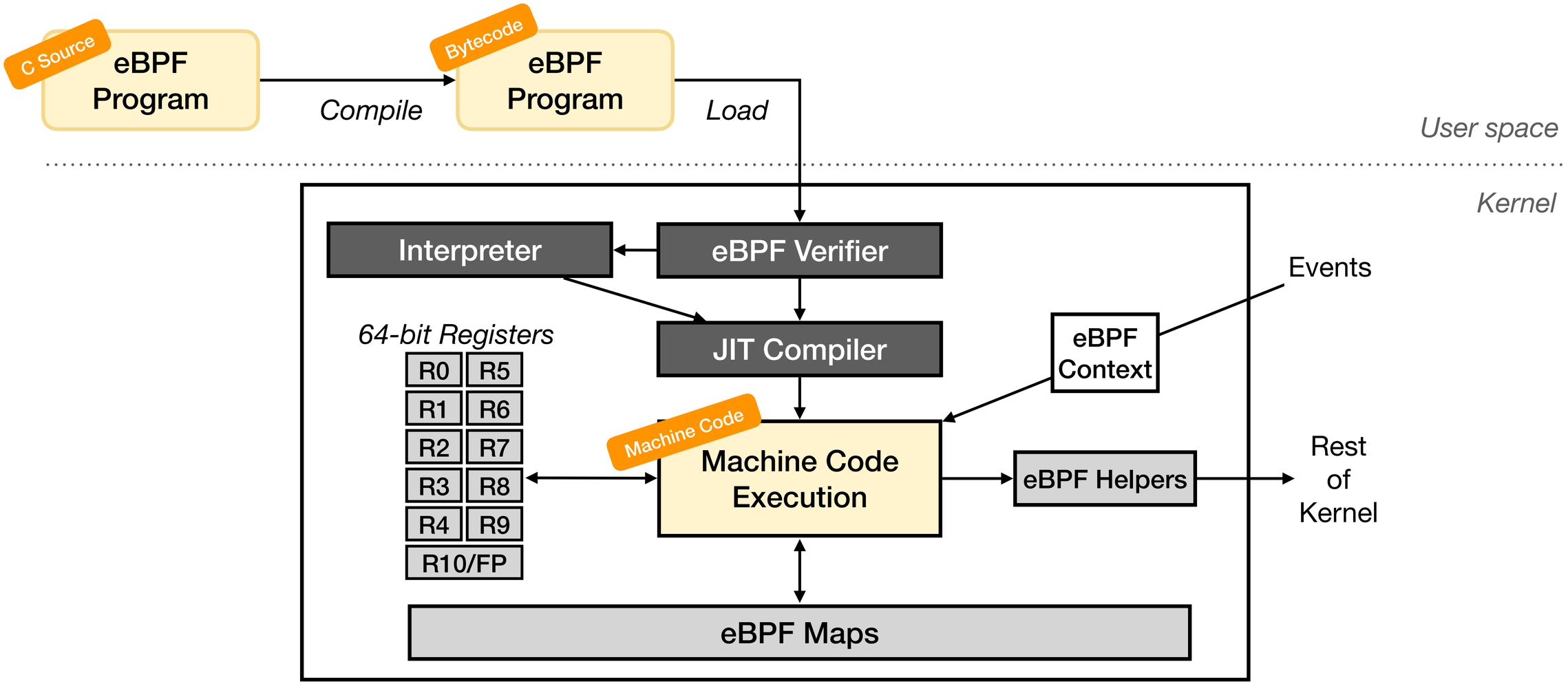
(Example)

What is eBPF?

Origin and Evolution

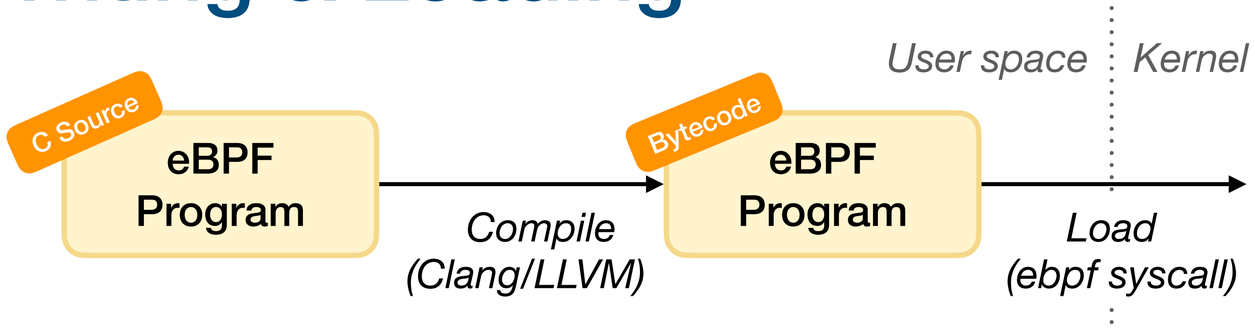
- **Berkeley Packet Filter (BPF)** was first developed in 1992.
  - A limited virtual machine for efficient packet filters.
  - Tools like tcpdump used the original BPF to capture and filter network packets.
- In 2013, a major redesign was proposed, and in 2014, the extended version (**eBPF**) was officially integrated into the Linux kernel.
  - This redesign transformed BPF from a packet-filtering mechanism into a general-purpose execution engine within the kernel, enabling innovative solutions to real-world performance, networking, and security challenges.

How does it work?



eBPF Program Workflow

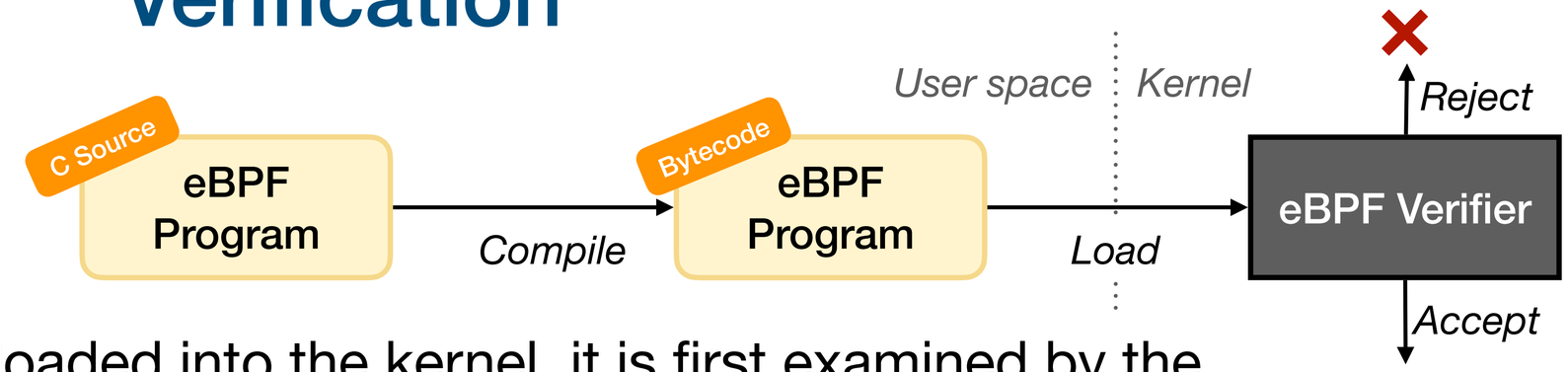
Writing & Loading



- An eBPF program is usually written in **(restricted) C code**, defining the program logic, the hooks to be attached, and the maps used.
- As the kernel expects eBPF programs in **bytecode** form, the C source code is compiled into a set of instructions that operate on virtual eBPF registers. This process is usually done using compilers such as Clang and LLVM.
- The eBPF bytecode is then loaded into the desired hook via the **ebpf syscall**. eBPF libraries facilitate this process.

eBPF Program Workflow

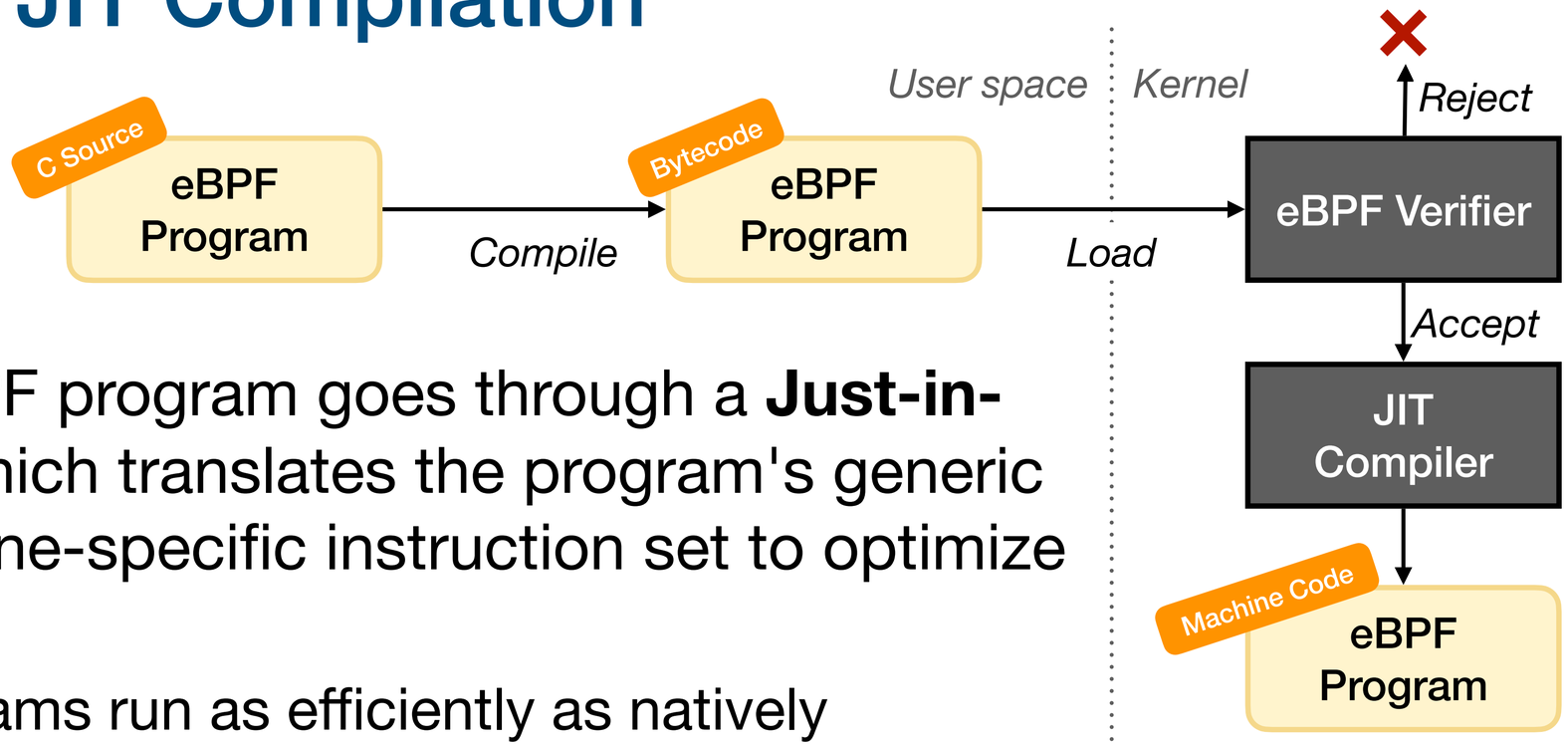
Verification



- When an eBPF program is loaded into the kernel, it is first examined by the **eBPF Verifier** to ensure it is **safe to run**. It does this in two steps:
  - It first performs a **static Control Flow Graph analysis** to identify unreachable code, then **simulates execution** to track register/stack states across all paths.
- Safety Guarantees (some examples):
  - **Program termination** (no infinite loops)
  - **Memory safety** (no out-of-bounds access)
  - **Type safety** (no pointer misuse)
  - **Bounded execution** (1M instruction limit)

eBPF Program Workflow

JIT Compilation

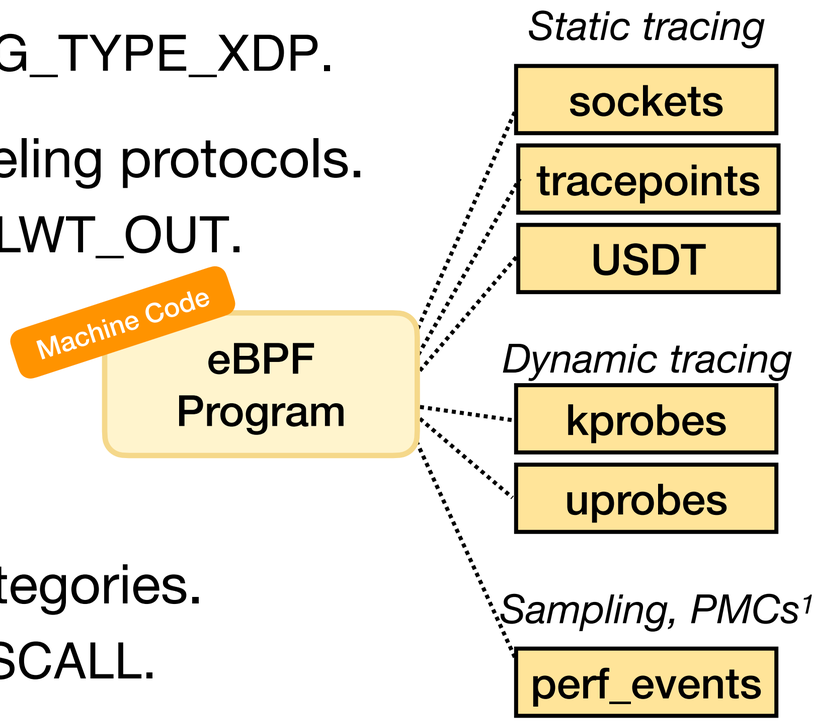


- Once accepted, the eBPF program goes through a **Just-in-Time (JIT) Compiler**, which translates the program's generic bytecode into the machine-specific instruction set to optimize execution speed.
  - This makes eBPF programs run as efficiently as natively compiled kernel code or as kernel modules.
  - The conversion to native machine instructions happens just once, when the program is loaded into the kernel.

eBPF Program Workflow

Program Types

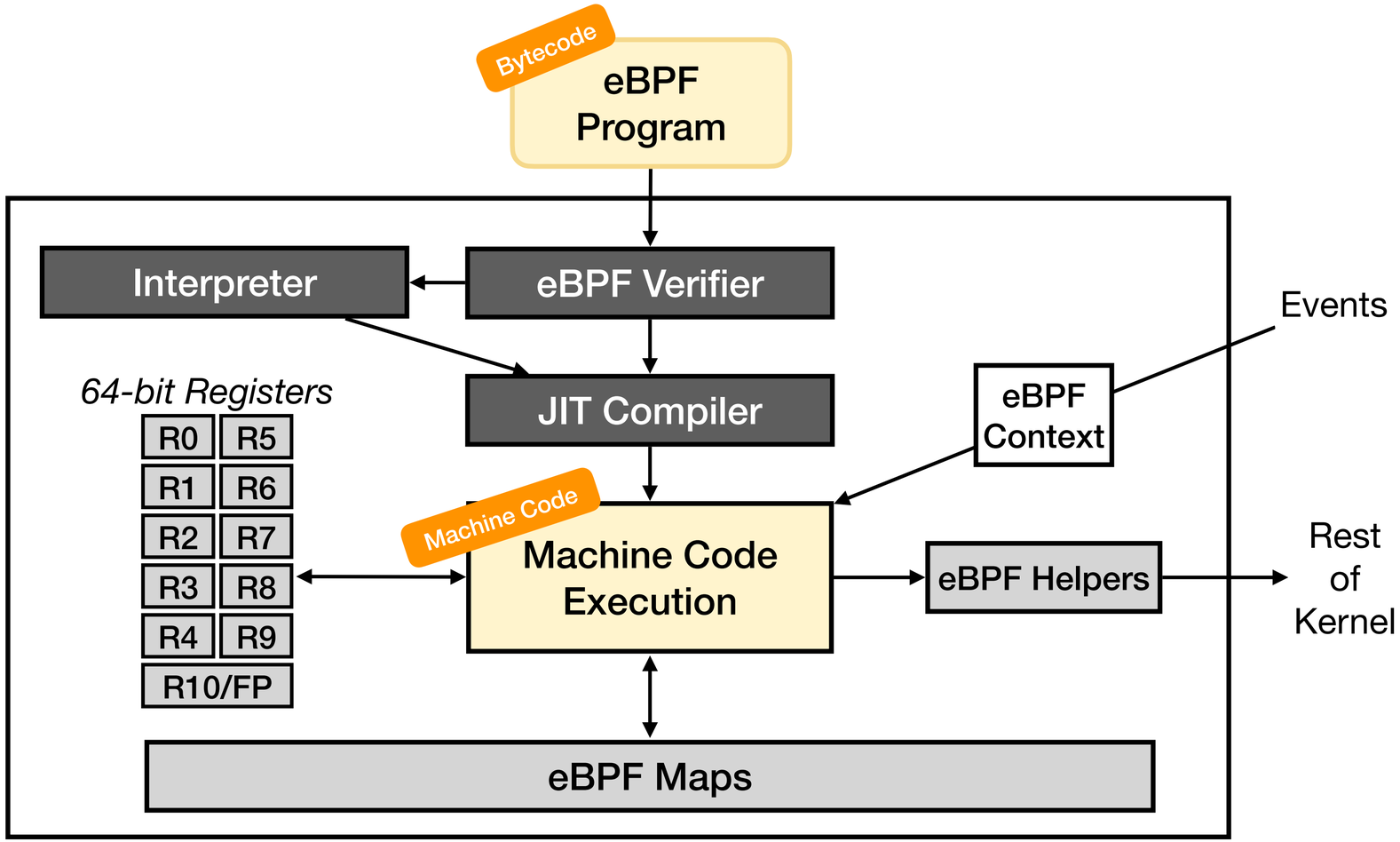
- **Tracing**: triggered by tracing events from the kernel or userspace.
  - Examples: BPF_PROG_TYPE_KPROBE, BPF_PROG_TYPE_TRACEPOINT.
- **Network**: triggered by network events.
  - Examples: BPF_PROG_TYPE_SOCKET_FILTER, BPF_PROG_TYPE_XDP.
- **Lightweight tunnel**: to implement custom lightweight tunneling protocols.
  - Examples: BPF_PROG_TYPE_LWT_IN, BPF_PROG_TYPE_LWT_OUT.
- **cGroup**: triggered by events of cGroups.
  - Examples: BPF_PROG_TYPE_CGROUP_SKB, BPF_PROG_TYPE_CGROUP_SYSCTL.
- **Misc**: with specialized purposes that do not fit the other categories.
  - Examples: BPF_PROG_TYPE_LSM, BPF_PROG_TYPE_SYSCALL.



[1] PMCs: Performance Monitoring Counters

eBPF Program Workflow

Machine Code Execution

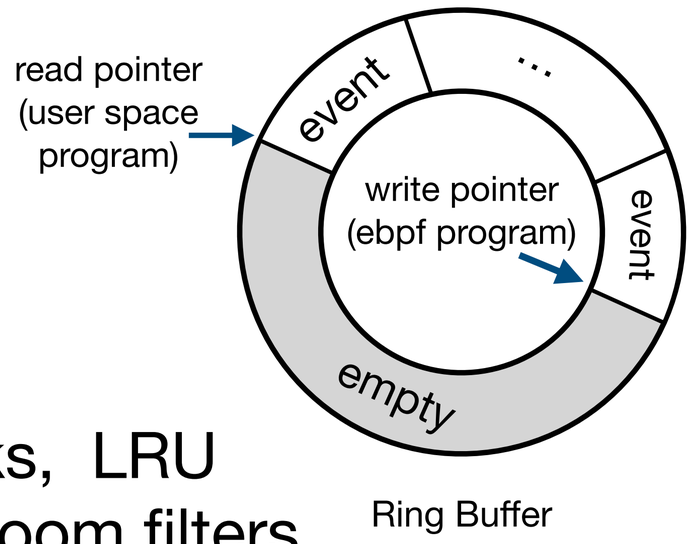


eBPF Maps

- **eBPF Maps** are in-kernel data structures (mainly key–value stores) used to persist and share data among multiple eBPF programs or between eBPF programs and user-space applications.
- Typical Use Cases:
  - Store configuration data from the user space for retrieval by an eBPF program.
  - Share state between different eBPF programs.
  - Export results or metrics from an eBPF program to the user space for processing/visualization.

eBPF Maps

Types



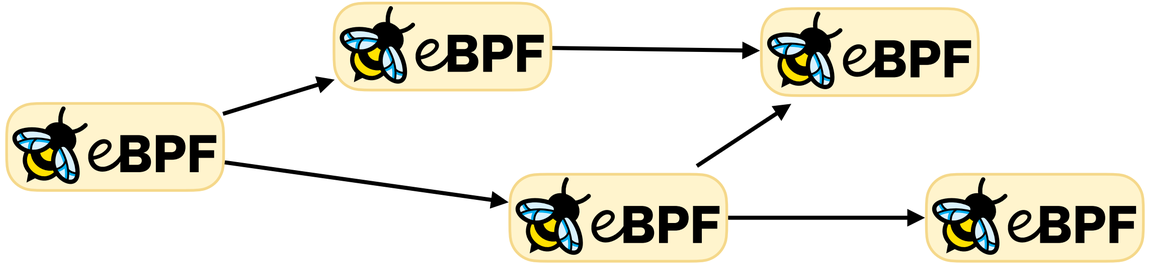
- **Generic Data Structures**: Arrays, Hash tables.
- **Operation-Optimized Maps**: FIFO queues, FILO stacks, LRU (Least Recently Used), LPM (Longest Prefix Match), bloom filters.
- **Object-Specific Maps**: Sockmaps (hold network sockets as value) and devmaps (hold references to network devices).
- **Per-CPU Maps:** maintain a separate instance per CPU core.
- **Ring Buffers** allow streaming large amounts of data across the kernel-user boundary.

eBPF Helper Calls

- **Helper functions** are functions defined by the kernel that can be invoked from eBPF programs.
  - Allow eBPF programs to interact with the kernel as if calling a function.
- Some examples:
  - **Generate random numbers** (bpf_get_prandom_u32)
  - **Get current time** (bpf_ktime_get_ns)
  - **eBPF map access** (bpf_map_lookup_elem, bpf_map_update_elem, bpf_map_delete_elem…)
  - **Get process/cgroup context** (bpf_get_current_pid_tgid, bpf_get_current_comm, bpf_get_current_cgroup_id, …)
  - **Manipulate network packets and forwarding logic** (bpf_skb_load_bytes, bpf_skb_change_proto, bpf_clone_redirect, …)

Function and Tail calls

- eBPF programs are composable by using function and tail calls.
  - **Function calls** allow defining and calling functions within an eBPF program.
    - Enable code reuse and reduce program size by avoiding inlining.
  - **Tail calls** can call and execute another eBPF program, replacing the execution context (similar to the exec syscall).
    - Allow chaining programs, splitting logic into independent components, and making eBPF programs modular and composable.



eBPF CO-RE

- **CO-RE (Compile Once – Run Everywhere):** Allows BPF programs to be compiled once into bytecode and then executed on multiple systems.
  - No need to install BPF compilers (LLVM/Clang) on every target, useful for space-constrained embedded Linux.
  - Avoids the runtime CPU/memory overhead of compiling programs whenever a BPF tool runs.
- **BTF (BPF Type Format):** Metadata format encoding debugging information for BPF programs, maps, and more.
  - Enables CO-RE relocations: the capability to adapt eBPF programs to the target kernel's data structure layout, even if that layout differs from the one used to compile the code.
  - Supports dumping human-readable kernel structs (via *bpftool*).

eBPF Development

Toolchains and Libraries

- An eBPF-based application typically consists of:
  - **eBPF programs** that run in the kernel
    - Mostly in C (Aya also supports Rust)
  - **user space code** that manages and interacts with eBPF programs
    - C/C++ (libbpf), Python/Lua (BCC), Golang (ebpf-go/libbpfgo), Rust (Aya/libbpf-rs)

- **Toolchains**: a set of tools to write, compile, load, and debug eBPF programs. Typically includes a compiler, loaders, helpers, and debugging utilities.
  - Examples: BCC, Bpftrace

- **Libraries**: provide APIs that user-space programs can use to load and interact with eBPF programs.
  - Examples: Libbpf, ebpf-go,libbpfgo, Aya, libbpf-rs, …

eBPF Development

Toolchains and Libraries

- **Libbpf** is the reference library for eBPF development.
  - Developed and maintained as part of the upstream Linux kernel.
  - Provides an eBPF loader that processes LLVM-generated eBPF ELF object files and loads them into the kernel.
  - Enables auto-generated BPF skeletons that simplify loading, attaching to, and interacting with BPF programs from user space.
  - Supports CO-RE.

eBPF Limitations

- **Restricted Kernel Access**
  - eBPF programs cannot call arbitrary kernel functions.
  - Limited to approved eBPF helper functions.
- **Execution Constraints**
  - No infinite loops (must always terminate).
  - Instruction limit: 1M since Linux 5.2.
- **Memory Constraints**
  - eBPF stack size is limited to MAX_BPF_STACK (set to 512 bytes).
  - Use BPF maps for storage instead of the stack.
- **Potential Data Loss**
  - When using ring buffers (which are size-limited), if the user space program does not consume data fast enough, new events may be dropped.

Adoption of eBPF

In Industry

- **Observability**
  - **Google**, **Cloudflare**, **Red Hat** – Performance monitoring & observability
  - **Netflix**, **Android** – Network usage
  - **Microsoft** – Kubernetes observability & process inspection
- **Networking**
  - **Meta** – Packet processing & load balancing
  - **Alibaba**, **IKEA**, **Red Hat** – Cloud/private cloud networking & load balancing
  - **Samsung**, **Android** – Networking in Android devices & traffic management
- **Security**
  - **Google** – Security auditing
  - **Cloudflare** – Network security
  - **Apple** – Kernel security monitoring

More examples at: https://ebpf.io/case-studies/.

Adoption of eBPF

In Academia

- eBPF has enabled research across multiple domains. Not only in observability, networking, and security, but also in OS optimizations, protocol design, and improving eBPF itself.

- **Operating Systems**
  - *cache_ext: Customizing the Page Cache with eBPF.* ACM SOSP, 2025.
  - *Syrup: User-Defined Scheduling Across the Stack*. ACM SOSP, 2021.

- **Storage Systems**
  - *λ-IO: A unified IO stack for computational storage.* USENIX FAST, 2023.
  - Extension Framework for File Systems in User space. USENIX ATC, 2019.

- **Distributed Systems**
  - *DINT: Fast In-Kernel distributed transactions with eBPF.* USENIX NSDI, 2024.
  - *Electrode: Accelerating distributed protocols with eBPF.* USENIX NSDI, 2023.

- **eBPF Verifier**
  - *Validating the eBPF verifier via state embedding.* USENIX OSDI, 2024.
  - *Verifying the verifier: ebpf range analysis verification.* Springer CAV, 2023.

More examples at: https://pchaigno.github.io/bpf/2025/01/07/research-papers-bpf.html.

More Information

- Brendan Gregg. **BPF Performance Tools**. Addison-Wesley, 2020.
- Liz Rice. **Learning EBPF: Programming the Linux Kernel for Enhanced Observability, Networking, and Security.** O'Reilly, 2023.

Questions?